

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

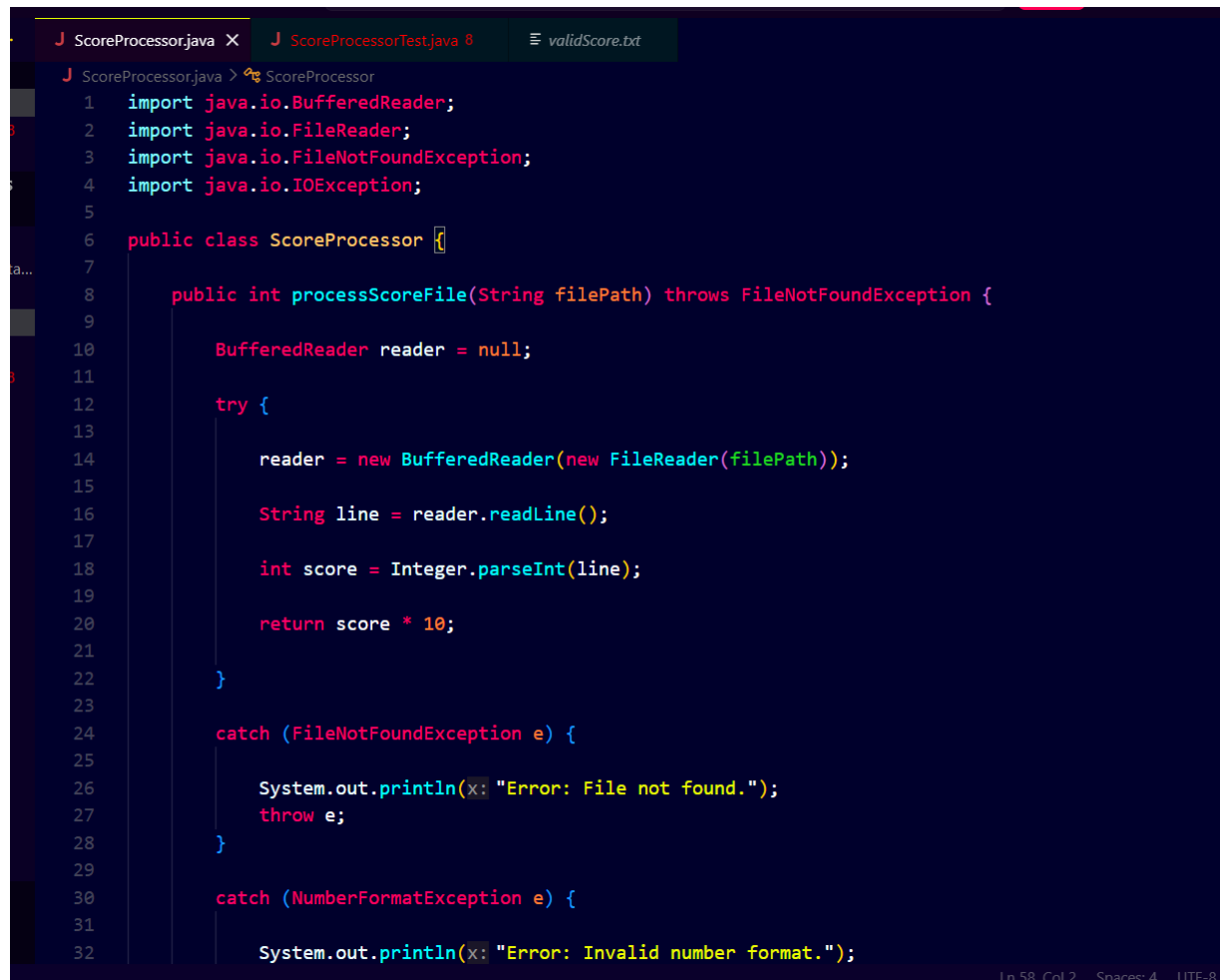
Year:2026

Question:

Assignment 18: Generally, handling external files is a common source of runtime errors. Files might be missing, or they might contain corrupted data. For this assignment, you will create a simple data utility class that reads an integer value from a text file, performs a calculation, and ensures all system resources are properly closed afterward—even if something goes wrong. Your program needs to read a numeric score from a file, multiply it by 10, and return the result. If the file does not exist, the system must catch that error and notify the user with a specific message. If the file exists but contains letters instead of a number, the system must handle that invalid data format gracefully. Finally, you must write basic automated tests to verify that your calculation works and that bad inputs are handled correctly. Implementation Rules: If choosing Java: Create a class named `ScoreProcessor`. Write a method `public int processScoreFile(String filePath)` that uses a try-catch-finally block (or a try-with-resources block) to open and read a file. Catch `FileNotFoundException` and `NumberFormatException` specifically, logging or printing a clear error message for each. Use the finally block to print a "File cleanup completed" message to the console. Write a JUnit 5 test suite with at least two test cases: one verifying a successful calculation with a valid file path, and one using `assertThrows` to check how the system reacts to a missing file. If choosing Python: Create a class named `ScoreProcessor`. Write a method `def process_score_file(self, file_path: str) -> int` that uses a try-except-else-finally block to open and read a file. Catch `FileNotFoundError` and `ValueError` specifically, printing a helpful error message for each. Use the else block to print "Data processed successfully", and use the finally block to print "File cleanup completed". Write a pytest suite with at least two test functions: one testing a successful calculation with a valid file, and one using `pytest.raises` to verify that a missing file is handled correctly. You must have the followings: Exception Handling & Structure: Correctly implementing the multi-catch structure (`FileNotFoundException` and `Invalid Format/Value errors`) and ensuring the cleanup block executes under all conditions. Core Logic

& Input Validation: Successfully reading the file content, parsing the text into a usable integer, and executing the required multiplication calculation. Unit Testing: Setting up a working test suite that uses correct framework assertions to validate both the happy path (successful calculation) and the error path (missing file).

Answer:



```
J ScoreProcessor.java X J ScoreProcessorTest.java 0 validScore.txt
J ScoreProcessor.java > ScoreProcessor
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.FileNotFoundException;
4 import java.io.IOException;
5
6 public class ScoreProcessor {
7
8     public int processScoreFile(String filePath) throws FileNotFoundException {
9
10         BufferedReader reader = null;
11
12         try {
13
14             reader = new BufferedReader(new FileReader(filePath));
15
16             String line = reader.readLine();
17
18             int score = Integer.parseInt(line);
19
20             return score * 10;
21
22         }
23
24         catch (FileNotFoundException e) {
25
26             System.out.println(x: "Error: File not found.");
27             throw e;
28         }
29
30         catch (NumberFormatException e) {
31
32             System.out.println(x: "Error: Invalid number format.");
```

Ln 58, Col 2 Spaces: 4 UTF-8

```
J ScoreProcessor.java X J ScoreProcessorTest.java 8 validScore.txt
J ScoreProcessor.java > ScoreProcessor
6 public class ScoreProcessor {
8     public int processScoreFile(String filePath) throws FileNotFoundException {
31
32         System.out.println(x: "Error: Invalid number format.");
33         throw e;
34     }
35
36     catch (IOException e) {
37
38         System.out.println(x: "Error reading file.");
39         return -1;
40     }
41
42     finally {
43
44         try {
45
46             if (reader != null) {
47                 reader.close();
48             }
49
50         } catch (IOException e) {
51
52             System.out.println(x: "Error closing file.");
53         }
54
55         System.out.println(x: "File cleanup completed");
56     }
57 }
58
```

```
View Go Run Terminal Help ← → ScoreProcessorProject   Sign In

... J ScoreProcessor.java J ScoreProcessorTest.java 8 X validScore.txt

J ScoreProcessorTest.java > ScoreProcessorTest > testSuccessfulCalculation()
1 import org.junit.jupiter.api.Test;
2
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.io.FileNotFoundException;
6
7 import static org.junit.jupiter.api.Assertions.*;
8
9 public class ScoreProcessorTest {
10
11     @Test
12     public void testSuccessfulCalculation() throws IOException {
13
14         String fileName = "validScore.txt";
15
16         FileWriter writer = new FileWriter(fileName);
17         writer.write(str: "abc");
18         writer.close();
19
20         ScoreProcessor processor = new ScoreProcessor();
21
22         int result = processor.processScoreFile(fileName);
23
24         assertEquals(80, result);
25     }
26
27     @Test
28     public void testMissingFile() {
29
30         ScoreProcessor processor = new ScoreProcessor();
31
32         assertThrows(FileNotFoundException.class, () -> {
```

Ln 20, Col 55 Spaces:

```
J ScoreProcessor.java  J ScoreProcessorTest.java  X  validScore.txt
J ScoreProcessorTest.java  >  ScoreProcessorTest  >  testSuccessfulCalculation()
9  public class ScoreProcessorTest {
25  }
26
27  @Test
28  public void testMissingFile() {
29
30      ScoreProcessor processor = new ScoreProcessor();
31
32      assertThrows(FileNotFoundException.class, () -> {
33
34          processor.processScoreFile("missingFile.txt");
35      });
36  }
37
38  @Test
39  public void testInvalidNumberFormat() throws IOException {
40
41      String fileName = "invalidScore.txt";
42
43      FileWriter writer = new FileWriter(fileName);
44      writer.write("ABC");
45      writer.close();
46
47      ScoreProcessor processor = new ScoreProcessor();
48
49      assertThrows(NumberFormatException.class, () -> {
50
51          processor.processScoreFile(fileName);
52      });
53  }
54  }
```

Ln 20, Col 55 Spaces: 4 UTF-8 CRLF {} Ja

Output:

```
PROBLEMS 8  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\gouri\OneDrive\Desktop\ScoreProcessorProject> java -jar junit-platform-console-standalone-1.10.2.jar --class
>>
>>
Test run finished after 87 ms
[      4 containers found      ]
[      0 containers skipped   ]
[      4 containers started    ]
[      0 containers aborted    ]
[      4 containers successful ]
[      0 containers failed     ]
[      3 tests found          ]
[      0 tests skipped         ]
[      3 tests started         ]
[      0 tests aborted         ]
[      3 tests successful      ]
[      0 tests failed          ]

WARNING: Delegated to the 'execute' command.
This behaviour has been deprecated and will be removed in a future release.
Please use the 'execute' command directly.
PS C:\Users\gouri\OneDrive\Desktop\ScoreProcessorProject>
```

TS
ava: Ready
Ln 20, Col 55 Spaces: 4